VitaSync Router – Anleitung

Stand 09.11.2026 · Branch `work/router-module` im Repository github.com/Afakani/VitaSync · Prototyp, nur synthetische Daten

> **Wichtig vorab:** VitaSync ist ein Wellness- und Präventionsprodukt, kein Medizinprodukt. Der Router beantwortet keine Fragen und gibt keinen medizinischen Rat. Er entscheidet nur, welchen Weg eine Nachricht nimmt. Bitte nur mit erfundenen Testfragen benutzen, nie mit echten Gesundheitsdaten.

1. Was der Router macht

Der VitaSync-Chatbot (FH Aachen, Team VC) beantwortet Fragen zu eigenen Gesundheitsdaten, gibt allgemeine Lifestyle-Tipps und leitet medizinische Fragen an Menschen weiter (Apotheke, Hausarztpraxis, 116117, im Notfall 112). Bevor eine Antwort entsteht, legt der Router für jede Nachricht genau einen Weg fest:

Route	Bedeutung
`DATA_QUERY`	Frage nach eigenen Werten, z. B. Schritte gestern
`HEALTH_QUESTION`	Allgemeine Gesundheitsfrage oder Lifestyle-Tipp
`COMBINED`	Eigene Daten plus allgemeine Info, z. B. Übersicht für einen Termin
`APP_HELP`	Frage zur App
`OUT_OF_SCOPE` + Sub-Label	Weiterleitung mit fester Vorlage, weil eine Regel (R1–R6) verletzt würde
`EMERGENCY` / `CRISIS`	Feste Notfall- bzw. Krisenvorlage (112 zuerst), ohne Sprachmodell
`CLARIFY`	Unklar: eine Rückfrage

Die Sub-Labels von `OUT_OF_SCOPE` und die Regel dahinter:

Sub-Label	Regel	Beispiel
`SCORE_REQUEST`	R1: kein Score, keine Ampel	„Gib mir eine Note für meine Gesundheit.“
`VALUE_EVALUATION`	R2: keine Bewertung von Werten	„Ist mein Ruhepuls zu hoch?“
`MEDICATION`	R4: keine Medikamentenempfehlung	„Wie viel Ibuprofen soll ich nehmen?“
`DIAGNOSIS_TRIAGE`	R4: keine Diagnose, keine Triage	„Was habe ich?“
`PHARMACY_SUMMARY`	R5: keine Deutung für die Apotheke	„Fass zusammen, was mit meinen Daten nicht stimmt.“
`COLOR_REQUEST`	R6: keine wertenden Farben	„Zeig meine Schritte rot, wenn sie zu niedrig sind.“

2. Wie er funktioniert

Grundidee: **Das Modell erkennt nur, was gefragt wird. Den Weg entscheidet fester Code.** So bleibt jede Entscheidung nachvollziehbar und testbar. Eine Nachricht läuft durch fünf Stufen (Funktion `route_message` in `apps/api/src/vitasync/features/chat/intake.py`):

1. CP1 Eingangsprüfung (`features/input_guard`): feste Wortmuster für Notfall, Krise und Prompt-Injection (`knowledge/rules/cp1_patterns.json`). Bei Notfall oder Krise kommt sofort die feste Vorlage, es wird kein Modell aufgerufen.

↓ kein Notfall, keine Krise

2. Erkennung (`features/recognition`): setzt 17 Merkmale (Flags), z. B. `asks_own_data` , `asks_value_evaluation` , `symptom_mentioned` , `acute_danger_signs` , plus einen Intent-Vorschlag mit Konfidenz. Zwei austauschbare Varianten:
· **SLM** (Standard): Qwen3-Embedding-8B macht aus dem Text einen Vektor mit 4096 Zahlen, eine logistische Regression je Flag rechnet daraus Wahrscheinlichkeiten. Schwellen: 0,35 für Sicherheits-Flags, 0,7 für drei vom Sicherheitsnetz abgedeckte Flags, sonst 0,5. Gewichte: `apps/api/models/slm_lr_qwen3-embedding-8b.json` .
· **LLM**: Qwen3-8B liefert die Flags als JSON; die Ausgabe wird gegen ein Schema geprüft (`contracts/llm/intent_flags.v1.schema.json`), ungültig heißt sicherer Rückfall.

↓ Flags

3. Regel-Pipeline (`features/rule_pipeline`): wendet R1–R6 auf die Flags an. Ergebnis ist eines von `crisis` , `emergency` , `block` (mit Sub-Label), `allow` oder `unclear` , mit maschinenlesbaren Gründen.

↓ Regel-Ergebnis

4. Routing (`features/routing`): wählt die Route aus Regel-Ergebnis und Modellvorschlag. Das Modell darf einen Weg nur vorsichtiger machen, nie lockern. Zu niedrige Konfidenz führt zur Rückfrage, Injection-Verdacht zum sicheren Rückfall.

↓ erlaubte Route

5. Router-Sicherheitsnetz (`features/safety_net` , Muster in `knowledge/rules/cp2_patterns.json`): feste Wortmuster nach dem Routing, z. B. „zu hoch“ oder „Note“. Kann eine erlaubte Route noch sperren, aber nie etwas freigeben. Im Code heißt es aus historischen Gründen „cp2“.

Das Ergebnis ist ein `RouteDecision` -Objekt mit u. a. `route` , `sub_label` , `stage` (CP1, ROUTER oder FALLBACK), `template` (Name der festen Vorlage, z. B. `TEMPLATE_OOS_VALUE_EVALUATION`), `reasons` und `confidence` . Nachrichtentexte werden nie gespeichert oder geloggt.

Policy: Einige Teamentscheidungen sind noch offen und als Schalter umgesetzt (`ROUTER_POLICY`). Leer heißt vorsichtig. Beispiel für die gelockerte Variante:
`symptom_with_data=allow,symptom_with_tip=allow,condition_named=general_plus_referral` .

3. Aufbau des Repositorys (für den Router)

Pfad	Inhalt
`apps/api/src/vitasync/core/`	Datenmodell (Routen, Flags, `RouteDecision`, `Policy`), Schnittstellen, Konfiguration
`apps/api/src/vitasync/features/`	input_guard, recognition, rule_pipeline, routing, safety_net, chat (Kette)
`apps/api/src/vitasync/adapters/`	Anbindung an Ollama (LLM und Embeddings)
`apps/api/models/`	Gewichte des SLM-Klassifikators
`knowledge/rules/`	Wortmuster für CP1 und das Sicherheitsnetz
`scripts/run_router_batch.py`	Router auf eine ganze Fragendatei anwenden
`benchmarks/rule_check/data/router/gpt/`	20 synthetische Fragensätze zu je 250 Fragen mit Soll-Route
`benchmarks/router_compare/`	Auswertung (`summarize_routes.py`) und bisherige Ergebnisse

Jeder Ordner hat eine `AGENTS.md` mit Zweck, Grenzen und Fallstricken. Gesamtüberblick: `docs/Architektur.md`.

4. Einrichten

Voraussetzungen

- Python 3.12
- Ollama (ollama.com), lokal installiert. Getestet auf einer RTX 2060; ohne GPU läuft es auch, nur langsamer.
- Für den Router allein ist keine Datenbank und kein Docker nötig.

Schritte (alle Befehle im Hauptordner des Repositorys)

```
git clone https://github.com/Afakani/VitaSync.git
cd VitaSync
git checkout work/router-module

# Modelle holen (einmalig)
ollama pull qwen3-embedding:8b      # für den SLM-Erkenner (Standard)
ollama pull qwen3:8b                # nur für den LLM-Erkenner

# Python-Umgebung
cd apps/api
python -m venv .venv
.venv/Scripts/python.exe -m pip install -r requirements.lock -e ".[dev]"   # Windows
# .venv/bin/python -m pip install -r requirements.lock -e ".[dev]"         # Linux, macOS
cd ../..

# Konfiguration
cp .env.example .env
```

> **Falle unter Windows:** Immer `http://127.0.0.1:11434` als Ollama-Adresse nutzen, nie `localhost`. „localhost" wird zuerst als IPv6 aufgelöst, Ollama hört nur auf IPv4, jeder Aufruf wartet dann rund 2 Sekunden extra. In `.env.example` ist es schon richtig eingestellt.

5. Benutzen

a) Tests (ohne Modell, ohne GPU)

```
cd apps/api
.venv/Scripts/python.exe -m pytest        # erwartet: 89 passed
```

b) Eine einzelne Nachricht routen

Ollama muss laufen. Im Hauptordner mit dem Python aus `apps/api/.venv` ausführen:

```
from vitasync.adapters.embedding import OllamaEmbedder
from vitasync.core.intent import Policy
from vitasync.features.chat import route_message
from vitasync.features.input_guard import InputGuard
from vitasync.features.recognition import SLM_MIN_CONFIDENCE, SlmRecognizer, SlmWeights
from vitasync.features.safety_net import SafetyNet

guard = InputGuard.from_file("knowledge/rules/cp1_patterns.json")
net = SafetyNet.from_file("knowledge/rules/cp2_patterns.json")
recognize = SlmRecognizer(
    OllamaEmbedder("qwen3-embedding:8b", "http://127.0.0.1:11434"),
    SlmWeights.from_file("apps/api/models/slm_lr_qwen3-embedding-8b.json"))
policy = Policy(min_confidence=SLM_MIN_CONFIDENCE)

d = route_message("Ist mein Ruhepuls von 85 zu hoch?", guard, recognize, policy, net)
print(d.route, d.sub_label, d.stage, d.template, d.reasons)
```

Mit dem LLM statt dem SLM tauscht man nur den Erkenner aus:

```
from vitasync.adapters.llm import OllamaIntentExtractor
from vitasync.features.recognition import LlmRecognizer
recognize = LlmRecognizer(OllamaIntentExtractor(model="qwen3:8b", base_url="http://127.0.0.1:11434"))
policy = Policy()   # Standard-Schwelle für das LLM
```

c) Einen ganzen Fragensatz laufen lassen und auswerten

```
apps/api/.venv/Scripts/python.exe scripts/run_router_batch.py ^
    benchmarks/rule_check/data/router/gpt/batch_01.tsv out/b01.csv --recognizer slm

apps/api/.venv/Scripts/python.exe benchmarks/router_compare/summarize_routes.py "SLM" out/b01.csv
```

Unter Linux und macOS `\` statt `^` für den Zeilenumbruch. Weitere Optionen: `--recognizer llm`, `--policy "symptom_with_data=allow,..."`, `--emb-cache datei.json` (Vektoren zwischenspeichern, zweiter Lauf braucht dann keine GPU). Eigene Fragen: Tab-getrennte Datei mit den Spalten `id`, `question`, `intent`, `sub_label`. Nie zwei Modellläufe gleichzeitig starten.

Die Auswertung zeigt: richtige Route, durchgelassene gefährliche Fragen („unter-blockiert", Ziel 0), verpasste Notfälle (Ziel 0), unnötig gesperrte harmlose Fragen („über-blockiert") und die Antwortzeit.

6. Messwerte (Stand 09.11.2026)

1.000 synthetische Fragen (Fragensätze 1, 2, 16 und 19), lokal, gelockerte Policy:

Variante	Richtige Route	Gefährliche durchgelassen	Notfälle verpasst	Harmlose gesperrt	Zeit je Frage
Nur SLM (Standard)	80,5 %	19 von 400	6 von 80	39 von 560	ca. 0,08 s
Nur LLM	80,2 %	8 von 400	4 von 80	85 von 560	ca. 5,7 s
Kaskade SLM + LLM, strengere Entscheidung gewinnt (nur simuliert)	78,6 %	2 von 400	0 von 80	79 von 560	ca. 5,4 s (geschätzt)

Zeiten auf einer RTX 2060, nicht unter sauberen Bedingungen gemessen; die Kaskaden-Zeit ist aus den Einzelzeiten geschätzt. Teamziel für die richtige Route: 90 % ± 2 (noch nicht erreicht).

7. Grenzen

- Prototyp, kein fertiges Produkt. Die Kaskade ist noch nicht eingebaut, nur ausgewertet.
- Die Wortmuster (CP1, Sicherheitsnetz) sind Teamentwürfe und nicht medizinisch geprüft. Kein Treffer heißt nicht „harmlos“.
- Die SLM-Gewichte wurden mit Vektoren eines API-Dienstes trainiert; lokal (quantisiertes Modell) weichen die Vektoren leicht ab. Ein Neutraining auf lokalen Vektoren ist geplant.
- Alle Testfragen sind synthetisch (von einem Sprachmodell erzeugt). Echte Nutzer formulieren anders.
- Der Router liefert nur den Weg und den Vorlagennamen; Antworttexte, Datenzugriff und Ausgabeprüfung sind andere Module.

Erstellt von Atakan Cubukcu, Team VC (FH Aachen). Router-Logik: Team-Router v0.5 (Tolga Toprakdöken), aufgeteilt in Module. Fragen gern direkt an mich.